TASK-1

Student Management System

Project Objective:
To develop a web application that allows users to manage student records easily, including adding, editing, deleting, searching, and organizing student information.

Target Users:
Students, teachers, tutors, and educational institutions that need to maintain basic student records.

Main Features:

- Add, edit, and delete student records.
- Search students by name.
- Filter students by course.
- Sort students by marks.
- Validate student information.
- Display total students and average marks.
- Save records using browser Local Storage.

Technologies Used:

- HTML5
- CSS3
- JavaScript
- Browser Local Storage

Brief Description:
The Student Management System is a responsive, browser-based application that manages student names, courses, and marks. It provides an interactive interface to organize records, search and filter students, sort marks, calculate statistics, and preserve data using Local Storage.

TASK-6

Project Documentation and Reflection

1. Project Overview

The Student Management System is a web application developed using HTML, CSS, and JavaScript. It helps users maintain and manage student records through a simple and interactive interface.

2. Features Implemented

- Add, edit, and delete student records.
- Search students by name.
- Filter students by course.
- Sort students by marks.
- Validate input fields.
- Display total students and average marks.
- Save records using Local Storage.

3. Technologies Used

HTML5, CSS3, JavaScript, and browser Local Storage.

4. JavaScript Concepts Used

Variables, functions, arrays, objects, conditional statements, loops, array methods, DOM manipulation, event listeners, form validation, and Local Storage.

5. Screenshots of the Project

Insert screenshots of the running application showing:

- Main dashboard and student form.
- Student records displayed in the table.
- Search, filtering, and sorting features.
- Editing or deleting a record.
- Statistics and input validation.

6. Challenges Faced

Managing student records, updating the table dynamically, preventing duplicate records while editing, and preserving information after refreshing the page were the main challenges.

7. Solutions Implemented

JavaScript functions were used to refresh the displayed records. Arrays and objects were used to manage student data, while Local Storage was used to preserve records between page refreshes. Input validation was added to reduce invalid entries.

8. What I Learned

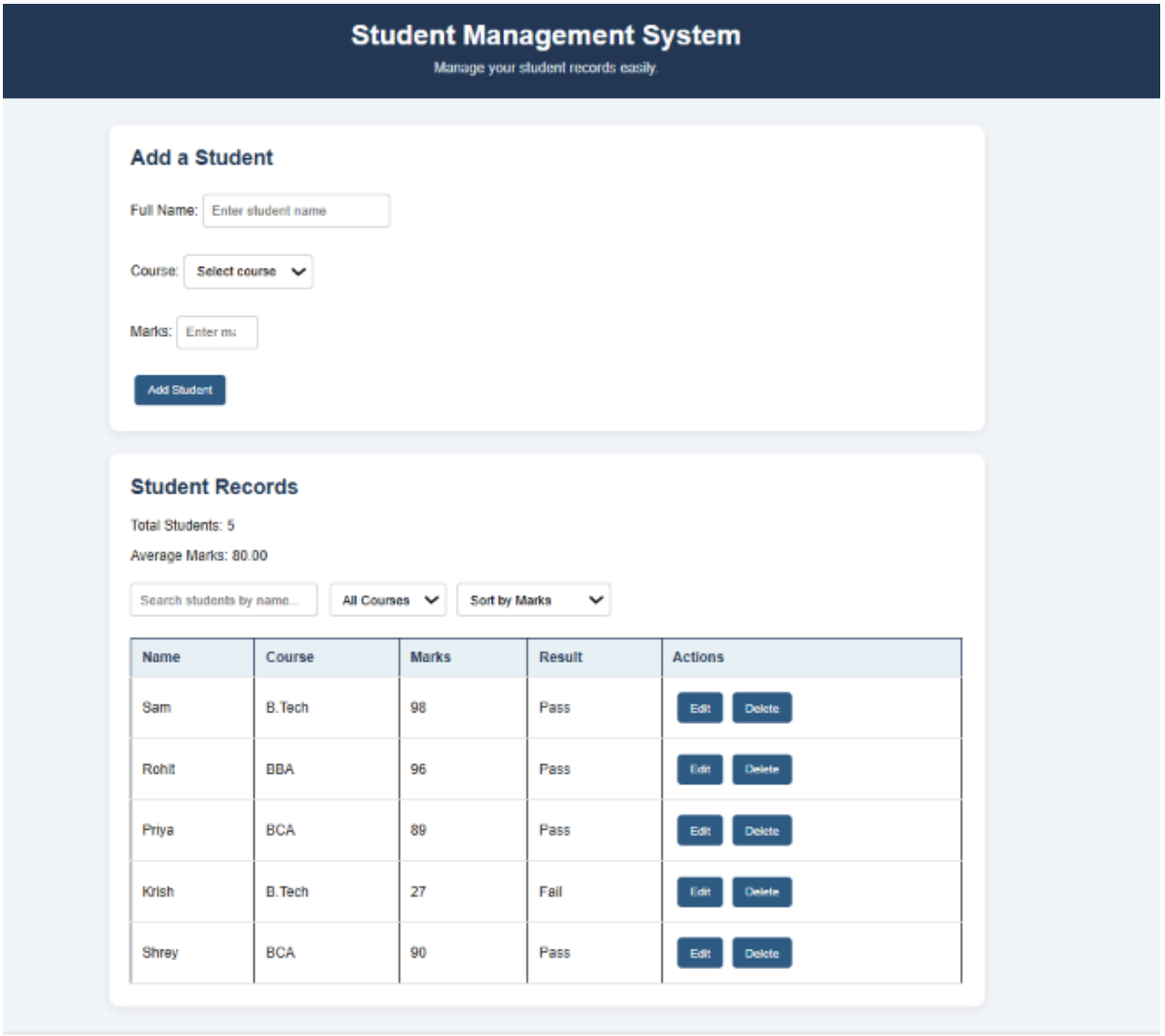
This project improved my understanding of JavaScript and how it works with HTML and CSS. I learned to manage data using arrays and objects, handle user interactions with event listeners, update webpage content through DOM manipulation, validate form inputs, and use array methods such as filter(), sort(), reduce(), and forEach(). I also learned the importance of testing and debugging an application.

9. Future Improvements

Future improvements could include unique student IDs, CSV export, user authentication, and database integration.

SCREENSHOTS:

Example with 5 student entries:



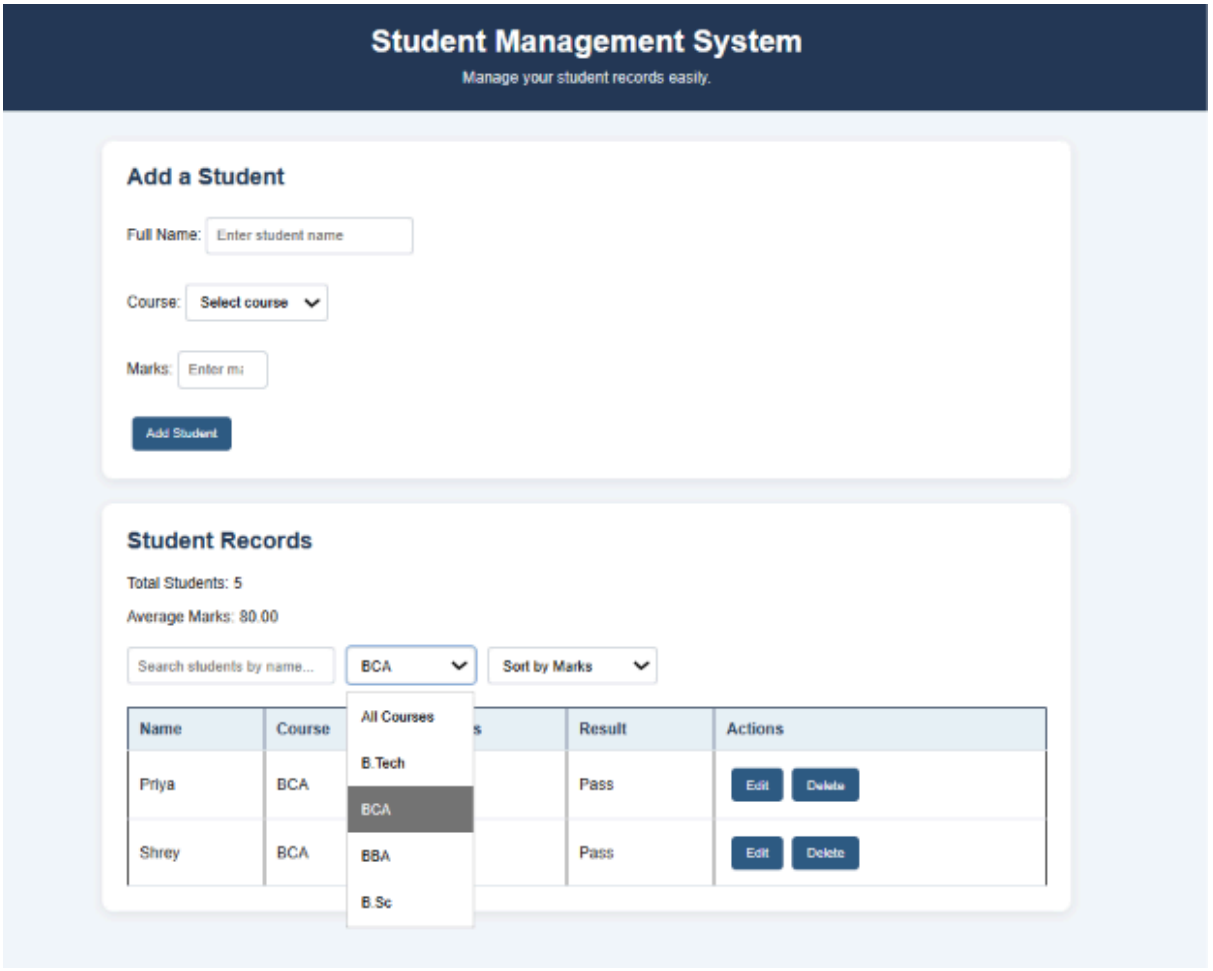
Student Statistics:



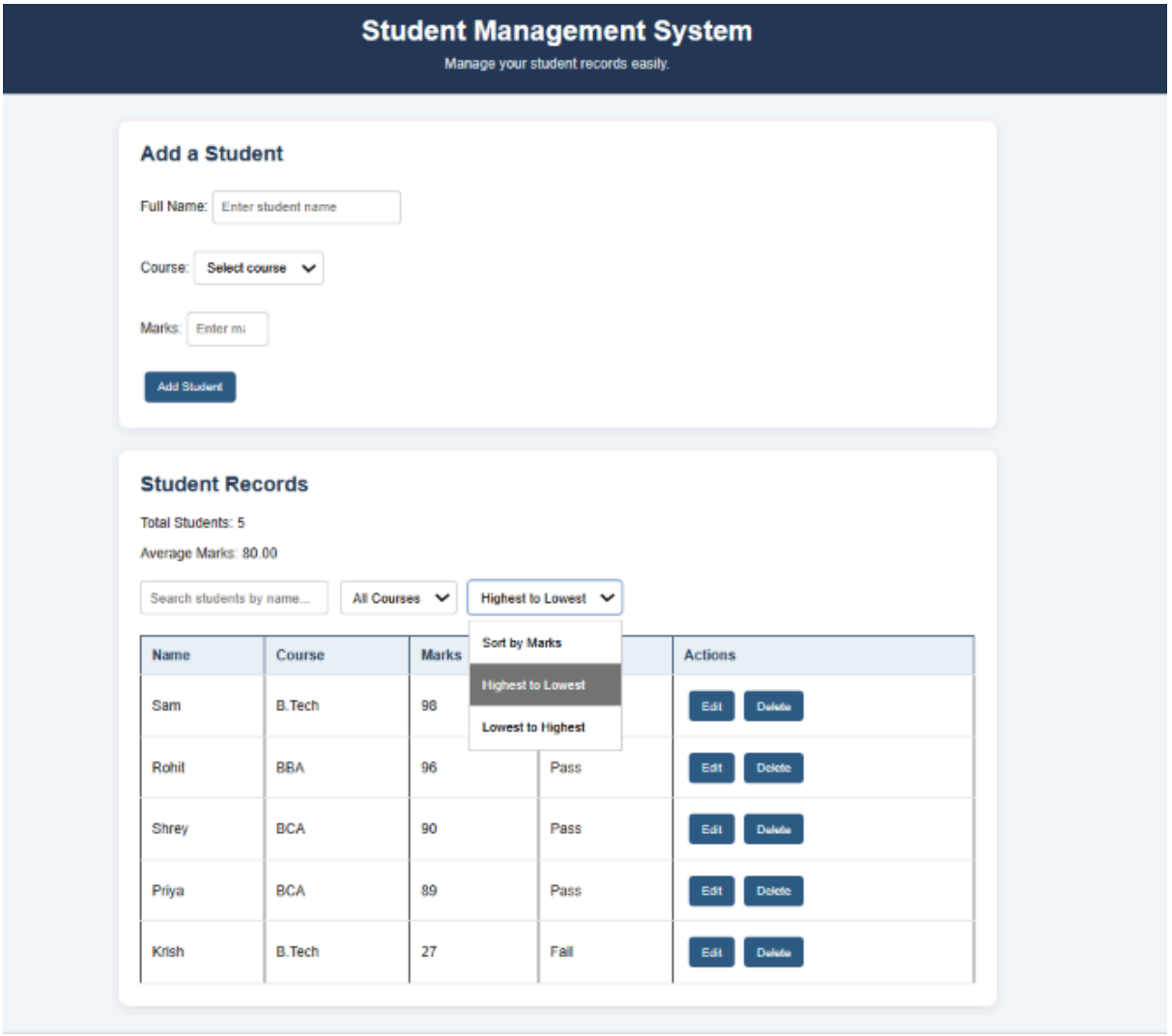
Searching students:



Selecting Course:



Sorting by marks: By Highest to Lowest / Lowest to Highest



Editing Student Details

